EFES Master Node: a Full Custom FPGA plus ESP32 Implementation of the AUSP Acoustic Protocol

Gioele Giunta s360501, Politecnico di Torino, LM-32
Course: Electronics for Embedded Systems

Academic Year 2025 / 2026

1 Introduction

AUSP is an acoustic communication protocol I created for my Bachelor thesis and later submitted to the IEEE IoT Magazine. Two devices talk using only a cheap speaker and a cheap microphone: packets are pairs of superimposed tones, one tone identifying the logical channel (the carrier) and one carrying the data symbol. The receiver moves the microphone signal to the frequency domain with an FFT and reads the two active tones as spectral peaks.

This version is a full custom FPGA implementation of the master side, paired with an ESP32. The FPGA is a Tang Nano 20K (Gowin GW2AR-18); I used Gowin EDA for synthesis and place & route, and GHDL for simulation. Everything on the FPGA is custom VHDL that I wrote the Wishbone interconnect, the SDRAM controller, two generic UARTs, three SPI masters (NOR flash, ADC, display), a GPIO, a two tone DDS/PWM generator, and the audio accelerator except three IP cores: the **PicoRV32** soft core that runs the firmware, the **rPLL** that generates the clocks, and the **Gowin FFT** core inside my accelerator.

The external hardware: a W25Q64 NOR flash (8 MB) for storage, a 2″ ST7789 display (240×320) on SPI, an electret microphone amplified by an MCP6022 and digitised by an MCP3201 12 bit SAR ADC, and an $8\,\Omega$ speaker driven from a PWM pin through a low side MOSFET. A single 12 V supply feeds the speaker directly and, through a DFR1202 buck, the 5 V input of the board. The ESP32 connects to the FPGA through two UART links and runs the protocol logic, the Wi-Fi link and a WebSocket web dashboard.

2 System overview

The system is a two node star: the master (FPGA + ESP32) is the master, a second, simpler ESP32 based node is the slave created 1 year ago for my thesis, and a browser dashboard is an optional viewer connected through a Cloudflare WebSocket. The only link between master and slave is acoustic: presence requests, identification, acknowledgements and aborts all travel as tones (master carrier 2014 Hz, slave carrier 2472 Hz, Section 13). Wi-Fi only carries dashboard traffic.

Inside the master, three units split the work by timing class. The FPGA does everything sample rate bound: it reads the ADC, computes the FFT, snapshots the spectrum and generates the PWM tones. The PicoRV32 soft core copies and decodes the spectrum, listens before it talks, runs the W25Q storage protocol through the FPGA and draws the display. The ESP32 does everything decision bound: protocol state machine, retries, dashboard. Table 1 lists the partition.

Table 1: Hardware/software partition.

Operation	Where	Type
ADC sampling, DMA to buffer, 512 pt FFT, SDRAM store, snapshot	FPGA fabric	hardware
Two tone generation (DDS sine sum, envelope)	FPGA fabric	hardware
Spectrum copy, tone decode, listen before talk	soft core	software
NOR storage protocol (W25Q), display rendering, health scan	soft core	software
Protocol state machine, framing, retries, Wi-Fi dashboard	ESP32	software

2.1 Board around the FPGA

Figure 1 is the board level schematic with the real parts. Three rails organise it: 12 V from a USB-C PD trigger feeds the speaker stage and the DFR1202 buck ($12\,\text{V} \rightarrow 5\,\text{V}$ into the board's VIN); the board itself only regulates and exposes 3.3 V for the digital peripherals, while the 5 V that supplies the analog (audio) front end is not a board generated rail but the buck's own output, the same node that feeds VIN. The 3.3 V does not reach the peripherals directly, it passes through an IRLZ44N MOSFET whose gate is held high by FPGA pin 80, so that output must stay at '1' or the NOR flash, the display and the LEDs all lose power.

3 The FPGA SoC

The GW2AR-18 is a 20k LUT FPGA whose package also contains an 8 MB SDR SDRAM die on dedicated pins. Figure 2 is the top level block diagram of the SoC.

3.1 Bus and address map

The interconnect (wb_interconnect.vhd) is a shared Wishbone bus with two masters and ten slaves. The masters are the PicoRV32 (M0) and the audio accelerator (M1). The slave is selected by decoding addr[31:27]: 5 bits, the PicoRV32 IP treats every address with addr[31:29]=000 as internal (ITCM, DTCM), so I needed to have 5 bits instead of 4 bits.
Table 2 is the map, with the essential register set of each slave.

Inside the interconnect the address and data lines are broadcast to every slave, but the strobe is not: a decoder reads addr[31:27] and a set of multiplexers send the handshake (cyc and stb) to the one matching slave only. That slave sees its stb and cyc go to 1 while all the others stay at 0, so the per slave strobe works like a chip select: only the addressed slave reacts and drives its read data back onto the shared dat_o. An address that matches no slave falls on a sink that never raises ack.

Table 2: Address map. The slave is selected by the top 5 bits addr[31:27]

Base addr[31:27]	**Slave**	**Block**	**Offset** addr[7:0]	**Register**
0x2000_0000	S6	character UART	0x00	tx byte (write) / pop rx FIFO (read, bit 8 = valid)
			0x04	enable
			0x08	disable
			0x0C	baud divider
			0x10	format
			0x14	status
0x2800_0000	S7	NOR command UART	same register map as S6 (second instance, pins 72/20)	
0x3000_0000	S5	audio accelerator	0x04	start
			0x08	stop
			0x0C	SDRAM base
0x3800_0000	S8	NOR flash SPI	0x00	rx byte
			0x04	tx byte (starts a transfer, clears done)
			0x08	CS bit
			0x0C	busy/done (non destructive)
0x4000_0000	S1	ADC SPI	0x00	sample (acks only when ready)
			0x04	start
			0x08	stop
			0x0C	clear ready
0x4800_0000	S0	SDRAM snapshot*	0 0 000100000 00	fetch FSM status
			0 0 000100001 00	samples counter
			0 0 000100010 00	triggers counter
			0 0 000100011 00	IRQ counter
			0 0 000100100 00	fill_cnt (frame counter)
			1 0 000000000 00	spectrum bin 0
			⋮	⋮
			1 0 111111111 00	spectrum bin 511
0x5000_0000	S2	speaker PWM	0x04	two 16 bit tones packed (f_1 in [15:0], f_2 in [31:16]); loads them and starts the DDS driving the PWM pin
			0x08	stop (PWM pin idles)
0x5800_0000	S9	display SPI	0x00	byte to shift out to the panel (MSB first, on SDA)
			0x04	drive the panel's DC / RST / CS lines
			0x08	busy (shifter still sending)
0x6000_0000	S3	LED PWM	same register map as S2	
0x7000_0000	S4	GPIO	0x00	drive pin (write) / read back (read)

Arbitration is static priority: when M1 asserts cyc & stb the mux gives it the bus, and M0 receives no ack and stalls until M1 is done. M1 wins because it collect one ADC sample every 21.3 μs: one MCP3201 SPI frame, 16 SCK $\times$ 54 clocks = 864 cycles at 40.5 MHz that make it's traffic tiny.

3.2 Masters slaves communication

Both masters the CPU (M0) and the audio accelerator (M1) drive the same Wishbone handshake. Every access carries the signals of Table 3.

Table 3: The bus signals of one access.

Signal	Driven by	Meaning
adr	master	address: top 5 bits pick the slave, low bits the register
we	master	1 = write, 0 = read
dat_i	master	data master → slave (on a write)
dat_o	slave	data slave → master (on a read)
cyc & stb	master	held high for the whole access ("I am accessing you now")
ack	slave	raised once the slave has taken or served the data

A write runs in four steps:

1. the master drives adr, sets we = 1 and puts the byte on dat_i;
2. it raises cyc & stb;
3. the slave whose addr[31:27] matches latches dat_i into the addressed register and raises ack;
4. the master sees ack and drops cyc & stb.

A **read** is identical but we = 0: the slave drives the data onto dat_o and raises ack, and the master samples dat_o on that cycle.

A master does not know how long a slave will take, so it holds cyc & stb and waits until ack comes back from the slave.

3.3 Clock tree

A 27 MHz external clock enters on pin 4. The internal rPLL divides by 2 and multiplies by 3, giving $f_{\text{clkout}} = 40.5$ MHz for the CPU, the bus and every peripheral on it; a second output clkoutp is the same 40.5 MHz shifted by $360°/16 = 22.5°$ (1.54 ns), and clocks only the SDRAM die to centre its data window (Section 5). The two PWM blocks use different clocks: the speaker PWM runs on the 40.5 MHz bus clock, so its registers need no clock crossing, while the LED PWM is clocked directly by the raw 27 MHz. Both divide by 256, so their carriers are 40.5 MHz/256 = 158.2 kHz and 27 MHz/256 = 105.5 kHz. Table 4 gives each block's divider.

Table 4: Clock and divider per block.

Block	Base	Divider	Result
SPI NOR flash	40.5 MHz	÷64	633 kHz SCK
SPI ADC	40.5 MHz	÷54	750 kHz SCK; 864 cycles/sample → $f_s = 46.875$ kHz
SPI display	40.5 MHz	÷6	6.75 MHz SCK, mode 3
UART (×2)	40.5 MHz	÷352	115 057 baud (0.1 % off 115200)
SDRAM controller	40.5 MHz	none	die clocked by clkoutp (+1.54 ns)
PWM speaker / LED	40.5 / 27 MHz	÷256	carriers 158.2 / 105.5 kHz

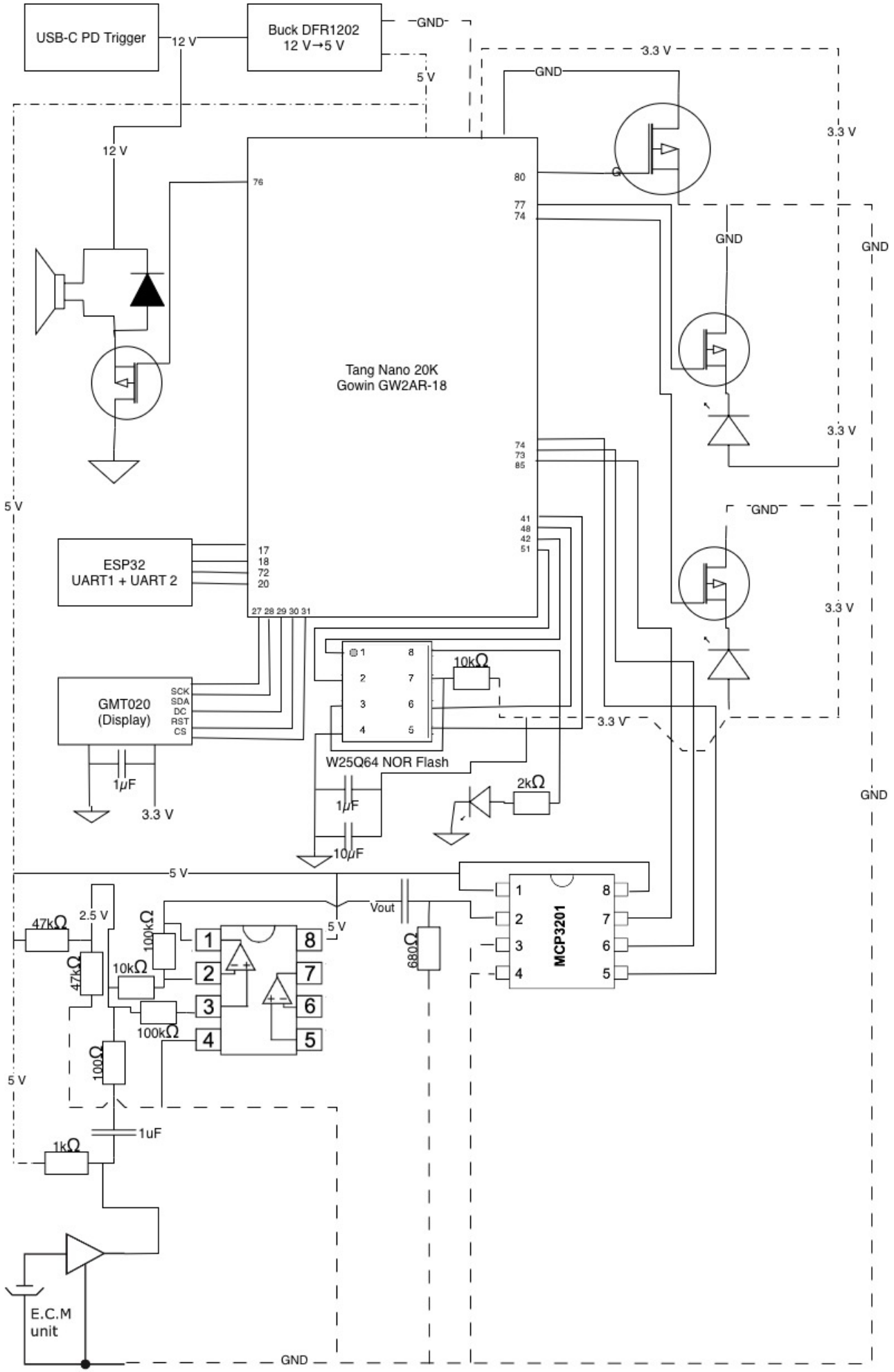

Figure 1: Board level schematic.

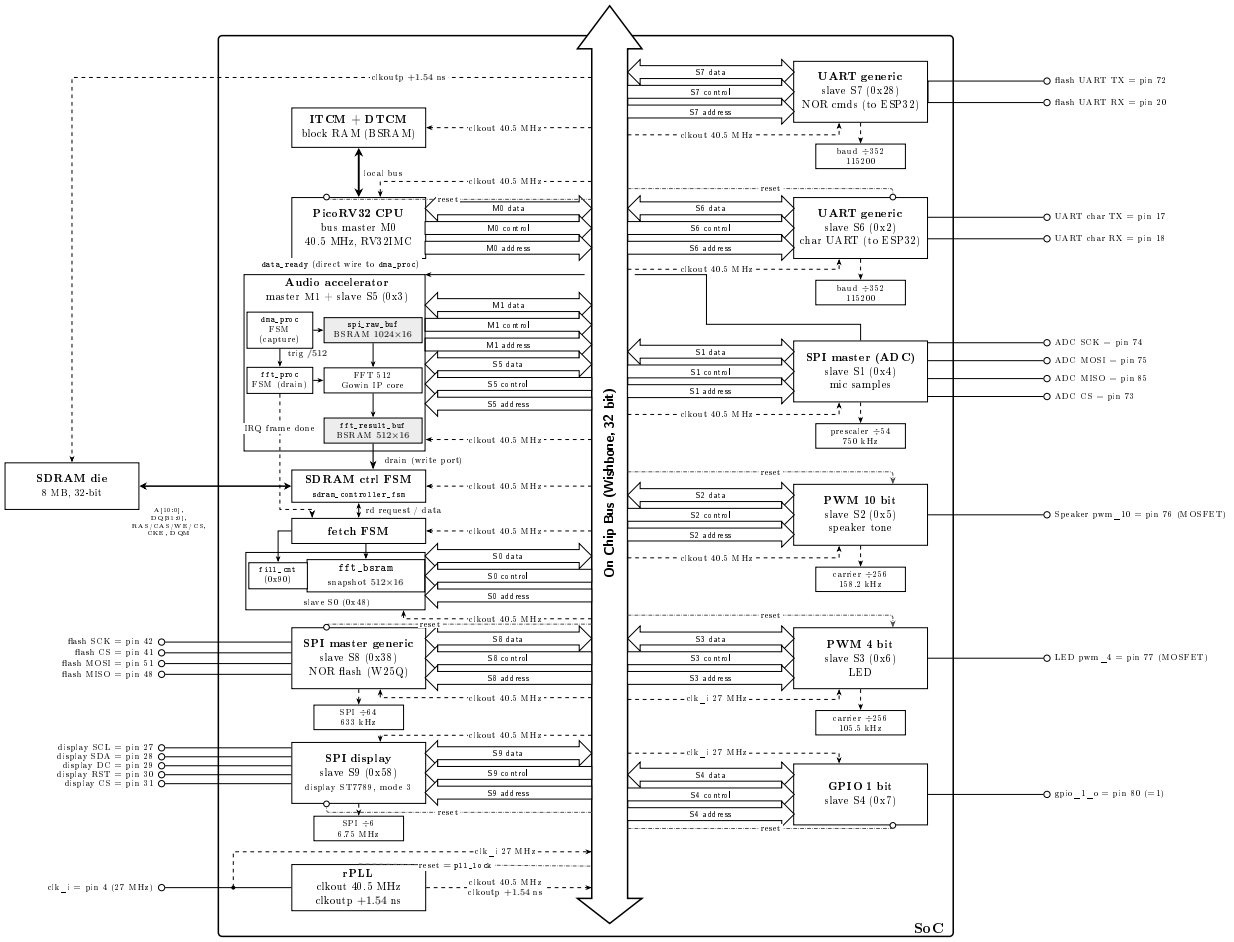

Figure 2: SoC top level block diagram.

4 Power management

The board runs on three rails (Figure 3). A USB-C PD trigger gives 12 V. The 12 V feeds two things: the speaker stage directly, and the input of a DFR1202 buck. The buck makes 5 V, and the 5 V feeds the board VIN and the analog front end. An on board regulator then makes 3.3 V from the 5 V, and the 3.3 V feeds the digital peripherals, the NOR flash, the display and the LEDs, through the GPIO switch described below.

Every switched load is driven low side: the control pin drives the gate of an N channel IRLZ44N MOSFET, the load sits between its supply and the drain, and when the gate goes high the MOSFET pulls the drain to ground so current flows. The gate is referenced to ground, so the 3.3 V logic level turns it on with no gate driver, and the speaker has a flyback diode for its inductance. The speaker and the LEDs are switched this way by their PWM pins.

The 3.3 V to the peripherals passes through one more IRLZ44N, gated by gpio_1 (pin 80). This is a soft power switch: the CPU can cut power to the NOR flash, the display and the LEDs by driving the pin low, so the pin must stay at '1' in normal use.

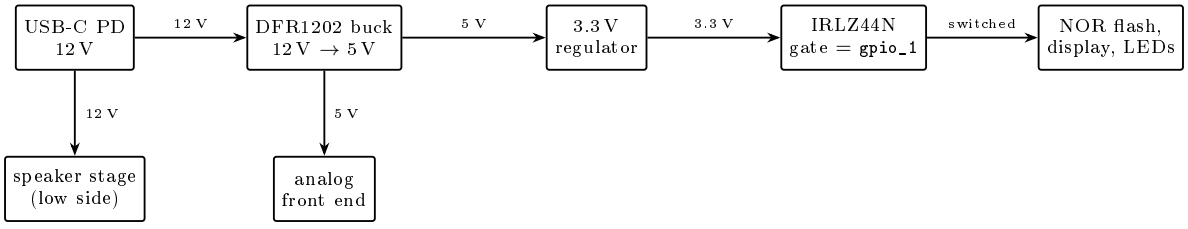

Figure 3: Power tree.

5 The SDRAM controller

The GW2AR-18 package contains an 8 MB SDRAM die, organized into 4 banks, and each bank is a grid of 2048 rows × 256 columns, where every cell holds one 32 bit word (so a row is 256 words = 1 KB, and

$4 \times 2048 \times 256 \times 4 = 8\,\text{MB}$). Based on the datasheet and the manufacturer's timing diagrams, the memory operates using burst transfers. A single read (rd_n) or write (wr_n) command, accompanied by a 21 bit address specifying the bank, row, and column, triggers a data burst of data_len+1 words. In my design, data_len is set to 26, resulting in a 27 word burst. The operation is managed by a few key control signals: busy_n indicates when the controller is ready to accept a new command request; rd_valid confirms that the current read word is valid data; and init_done signals that the power up initialization sequence is complete.

Figure 4 shows how one bank is addressed, and its two access delays.

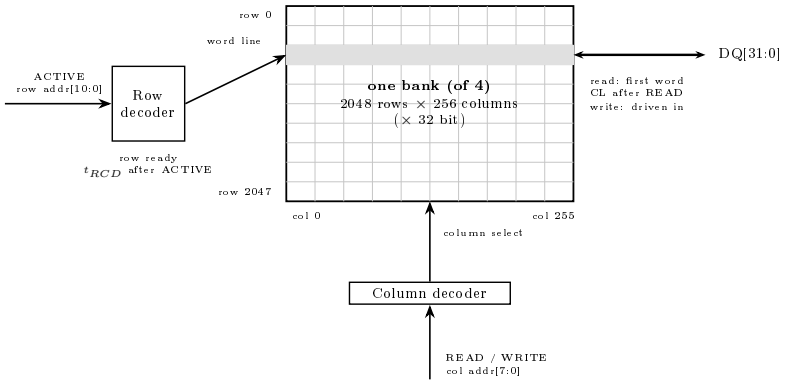

Figure 4: One SDRAM bank: 2048 rows by 256 columns, and how a row and a column are selected, with the two access delays.

5.1 The FSM

A combinational process computes next_state only, and a synchronous process registers the state, one shared counter tmr (cleared on every state change), and all outputs a Moore machine with registered outputs, where each JEDEC command (the standard that defines the SDRAM commands) is emitted exactly on the first cycle of its state (detected as next_state$\neq$curr_state) and NOP fills the rest. The mode register is programmed for CAS latency 3 (CL is the one timing I set myself; I use 3 because it only adds two cycles over CL 1 and changes nothing for the project constraints, and this way I am sure there is no problem on that side), sequential addressing and full page burst. This is why the column address goes out only once, with the READ or WRITE command: from there the die steps through the columns by itself, one per cycle and in order (sequential), so I never resend it. The burst length is how long the FSM stays in the data state, and the closing PRECHARGE ends it. The FSM also issues an AUTO REFRESH about every 7.8 μs: a counter asks for one every 312 cycles, served in S_IDLE. Table 5 lists the commands it issues.

Table 5: The SDRAM commands the FSM issues, and their code on the four control pins (active low): the FSM never sends the word, it drives this pattern for one cycle.

Command	$\overline{\text{CS}}\,\overline{\text{RAS}}\,\overline{\text{CAS}}\,\overline{\text{WE}}$	**Role**
ACTIVE	0 0 1 1	opens a row, copying it into the sense amplifiers
READ	0 1 0 1	reads a column from the open row
WRITE	0 1 0 0	writes a column into the open row
PRECHARGE	0 0 1 0	closes the open row, writing it back
AUTO REFRESH	0 0 0 1	refreshes the rows so they keep their charge
LOAD MODE REGISTER	0 0 0 0	sets CAS latency, burst type and length
NOP	0 1 1 1	does nothing, fills the waits between commands

I read the JEDEC instruction set from the SDRAM datasheet and turned each into the number of 40.5 MHz clock cycles the FSM has to hold before it issues the following command (Table 6). Those cycle counts are exactly the numbers written in the state graph below.

Table 6: SDRAM timings from the datasheet, in 40.5 MHz cycles.

Parameter	Role	Cycles	Wait
power up wait	stable clock before init	8192	202 μs
t_{RCD}	ACTIVE to READ/WRITE	3	74 ns
CL	READ to first valid word	3	74 ns
t_{RP}	PRECHARGE to next command	2	49 ns
t_{RFC}	AUTO REFRESH to next command	4	99 ns
t_{MRD}	LOAD MODE REG to next command	2	49 ns
t_{WR}	last write to PRECHARGE	2	49 ns
refresh	one AUTO REFRESH every	312	7.7 μs

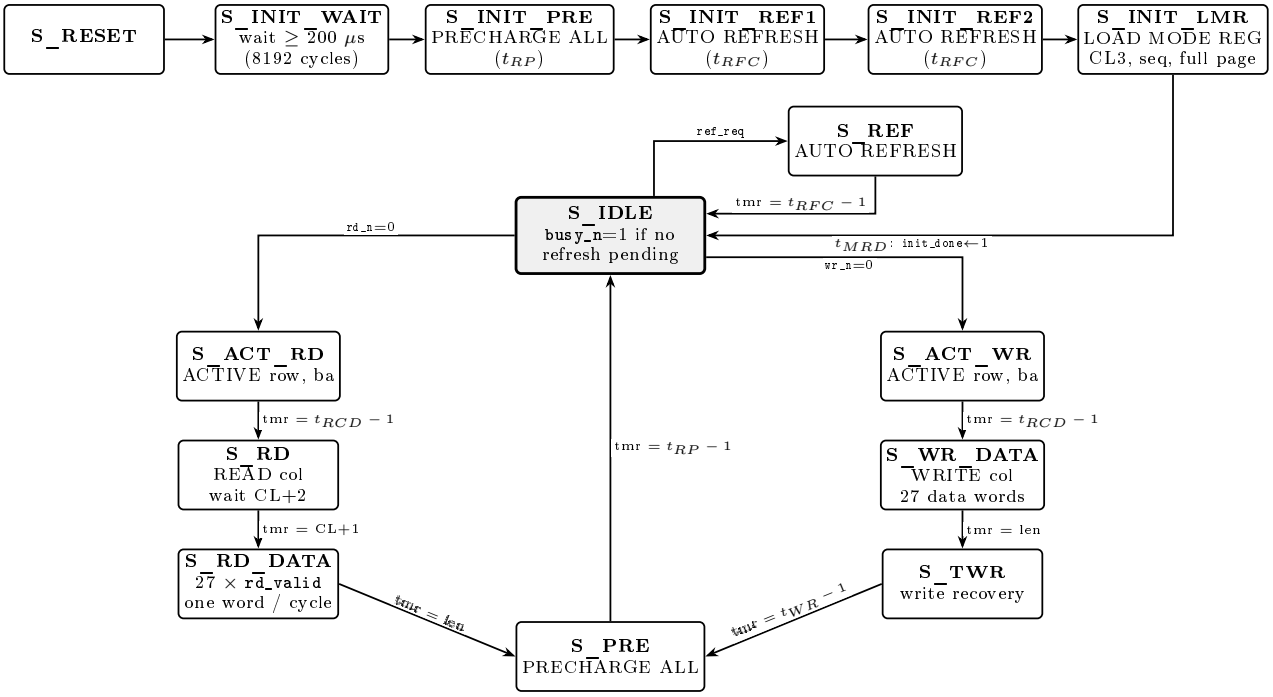

Figure 5: Complete sdram_controller_fsm state graph.

5.2 Bring up sequence

At power up the die must be initialised in a fixed order before it works. The FSM runs it once and then raises init_done:

1. wait 8192 cycles (202 μs) with CKE high;
2. PRECHARGE ALL, to put the four banks in idle;
3. two AUTO REFRESH commands;
4. LOAD MODE REGISTER (CL 3, sequential, full page).

This runs only at boot. After init_done the controller issues normal commands plus one AUTO REFRESH every 312 cycles. In the diagram the address line addr[10] (O_sdram_addr[10]) doubles as a control bit: on a PRECHARGE the chip reads addr[10]= 1 as "all four banks", which is why it is high during PRECHARGE ALL.

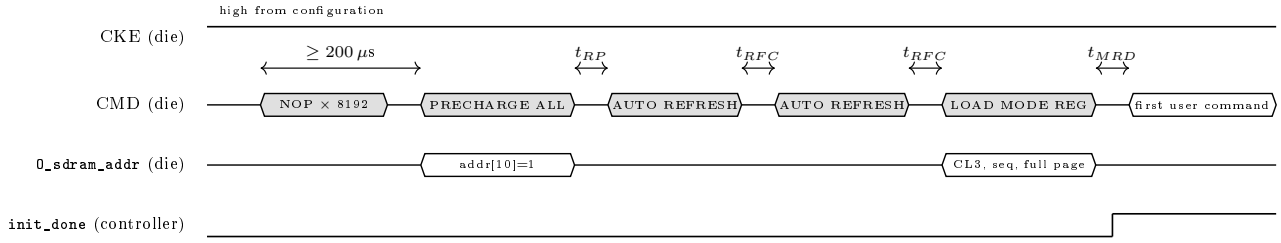

Figure 6: SDRAM bring up sequence.

5.3 Read path and clock phase

Figure 7 shows a read burst cycle by cycle. After a read request the FSM sends ACTIVE, then READ three cycles later (this wait is t_{RCD}), and three cycles after READ (the CAS latency CL) the die puts the first word on its data pins (the DQ bus). Both waits, and the others further down, are the JEDEC numbers listed in Table 6. The SDRAM has no valid line of its own, so the controller times the read by counting. The word passes through two flip flops inside the controller: the first one captures the data pins coming from the die, the second one holds that word ready. When the word reaches the second flip flop the FSM raises rd_valid, which tells that the word on O_sdrc_data is good. So rd_valid goes high CL+2 cycles after READ, one extra cycle per flip flop, and stays up for the 27 words, one per cycle.

Clock phase. On a read the die does not drive its data pins at once: the data comes a few nanoseconds after the die's clock edge (the access time t_{AC}) and then travels back over the board to the FPGA, so it arrives late. With the die on the same clock as the controller I was sampling too early, and on a few reads a bit came back wrong (0xFFFE instead of 0xFFFF). Feeding the die the 1.54 ns shifted clock (clkoutp) lets me sample in the middle of the data, and the errors stopped.

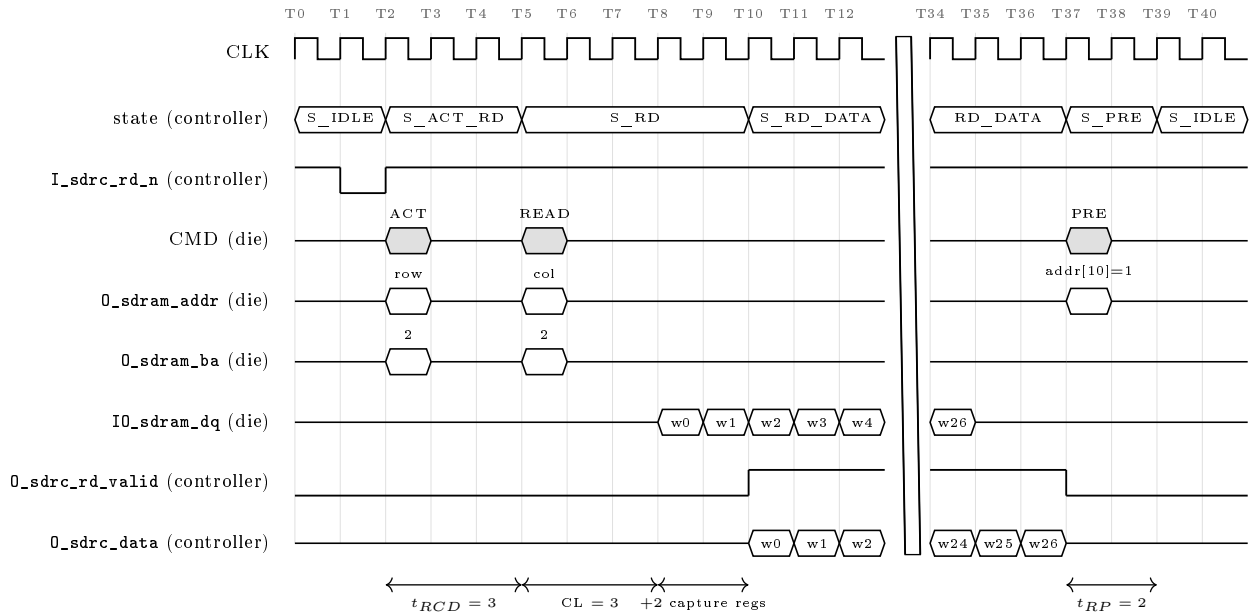

Figure 7: Read burst, cycle accurate.

5.4 Write path

On a write the FSM sends one word per cycle. The controller has to line the words up so that word 0 lands on the data pins in the same cycle as the WRITE command, which goes out three cycles after ACTIVE (the t_{RCD} wait of Table 6). It does this by pushing the words through a short chain of flip flops (wpipe1 → wpipe2 → wpipe3 → dq_out_r) that shifts every cycle (Figure 8). A write has no CL wait and no capture registers: here the controller is the source, so it drives the data on its own clock and only aligns it with the command. On a read the die is the source, so the data comes CL cycles later and must be recaptured, which is why a read is the slower of the two.

The alignment stage WR_ALIGN. On the board word 0 arrived one cycle early and was lost, shifting the whole burst by one bin. One extra flip flop (WR_ALIGN) puts it back in step; the simulation has no board delays and uses WR_ALIGN= 0.

How the spectrum is written. Figure 8 is one burst: 26 words go into 26 columns next to each other in one open row (the column goes up by one every cycle, the row stays the same). The whole spectrum is 20 bursts, one per row (rows 2 to 21 of bank 2). I use one row per burst to keep the addressing easy: the row is the burst number, and each bin goes to position burst×26+index. Only 26 of the 256 columns of a row are used, but the chip is 8 MB and there is plenty of time, so filling the rows would not help. Each drain starts with one extra warm up burst into row 1 that is thrown away, because the first access after a pause was the one that gave errors.

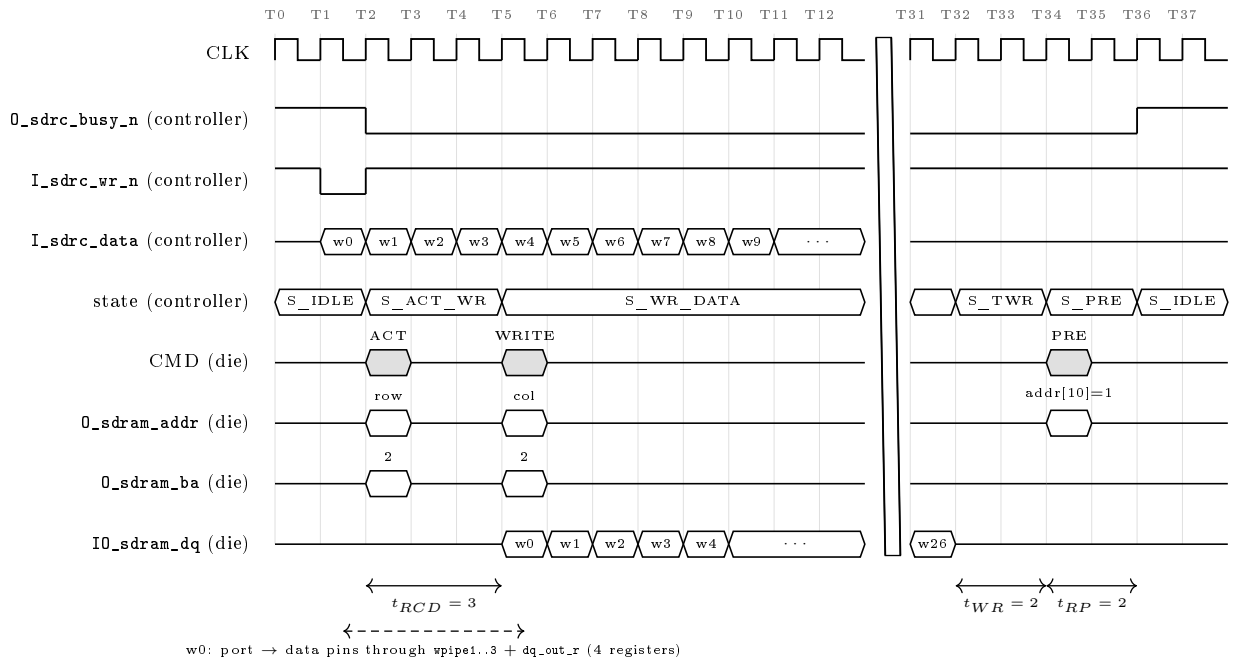

Figure 8: Streaming write burst, cycle accurate.

6 The audio pipeline: ADC → DMA → FFT → CPU

The receive path is two state machines running in parallel with a ping pong buffer between them, inside the block I call the audio accelerator (dma.vhd, Figure 9). The capture FSM (dma_proc, bus master M1) waits for the SPI master's data_ready_s flag (a direct wire, not the bus), reads the sample over the bus, clears that flag, and writes the sample into spi_raw_buf, a 1024×16 block RAM holding two 512 sample windows. The FFT FSM (fft_proc) takes the window just filled, feeds the Gowin FFT core, collects the 512 spectrum words into fft_result_buf, drains them to SDRAM through the dedicated write port and raises the frame IRQ. While one half fills, the other half is processed, so no sample is ever lost. The CPU starts the accelerator once at boot and never reads a raw sample. The tone decoding runs in firmware, which is the goal I wanted: do every decision in software.

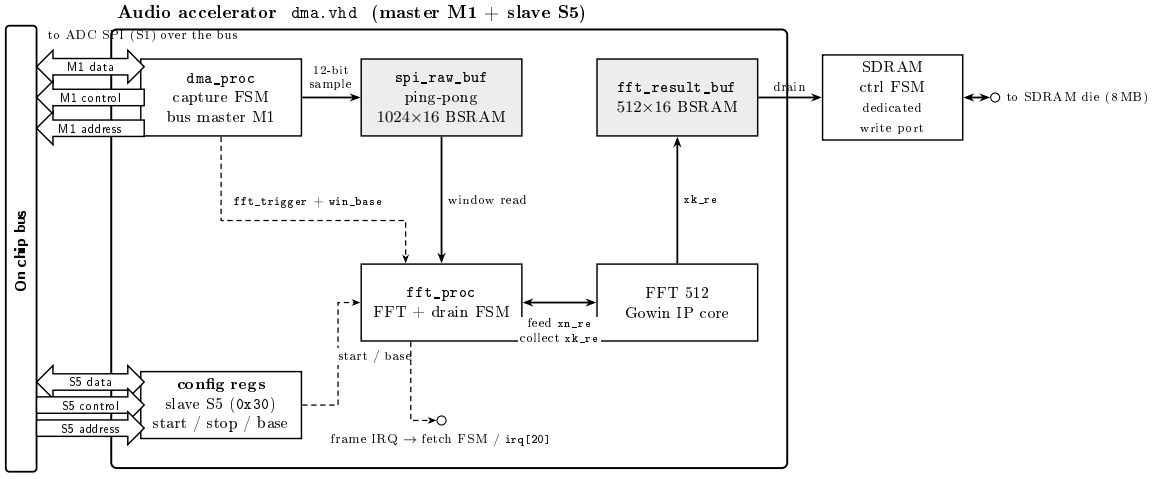

Figure 9: Audio accelerator block diagram.

6.1 The two state machines

Both are drawn as state diagrams. Each box is a state, with its name and what it does there (the register updates); each arrow shows the condition that moves the machine to the next state.

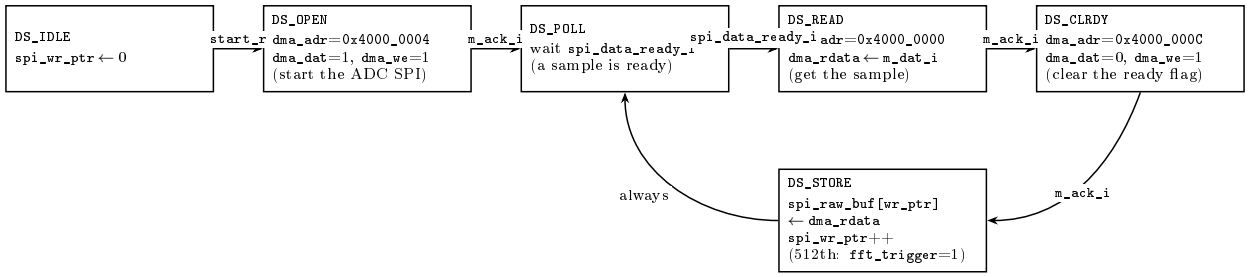

Figure 10: Capture FSM (dma_proc).

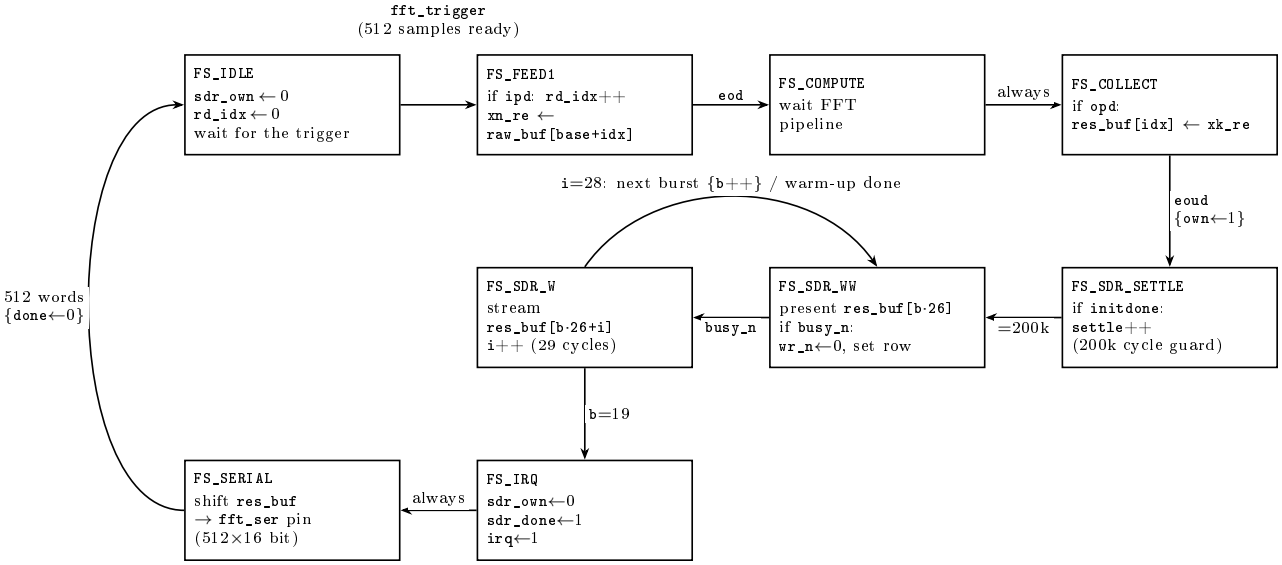

Figure 11: FFT and drain FSM (fft_proc).

6.2 The ADC SPI master

The analog front end (bottom of Figure 1) biases the electret capsule with a 47k/47k divider, amplifies it by $G = 1 + 100\text{k}/10\text{k} = 11$ in an MCP6022 non inverting configuration, and passes it through an RC low pass into the MCP3201. The filter uses a $680\,\Omega$ resistor with the 102 (1 nF) capacitors, so $f_c = 1/(2\pi \cdot 680\,\Omega \cdot 1\,\text{nF}) = 234\,\text{kHz}$; it only trims fast noise. A tight anti alias filter is not needed here because the protocol tones (2 to 4577 Hz) sit far below Nyquist ($f_s/2 = 23.4\,\text{kHz}$), and the electret and the amplifier already roll off well before it. The MCP3201 is a 12 bit SAR ADC that the FPGA reads over SPI with a dedicated master (spi_master.vhd); its internal block diagram is in Figure 12.

The master runs off the 40.5 MHz clock and makes one SCK every 54 clock cycles:

$$f_{\text{SCK}} = \frac{40.5\ \text{MHz}}{54} = 750\ \text{kHz}.$$

A frame is 16 SCK, so it takes $16 \times 54 = 864$ clock cycles $= 21.33\,\mu\text{s}$, and the sample rate is $f_s = 1/(21.33\,\mu\text{s}) = 46.875\,\text{kHz}$. The frame is simple: CS goes low, the ADC sends one null bit and then the 12 data bits MSB first, and after the data CS goes high for 2 SCK and the frame starts again. Nothing is sent to the ADC, so MOSI is unused.

The FSM has three states: S_SHIFT covers bit periods 0 to 13 with CS low (the 12 data bits are captured on periods 2 to 13), S_GAP covers periods 14 to 15 with CS high, and while the start register stays set the machine loops S_GAP→S_SHIFT with no dead cycle. When the frame ends data_ready_s goes to 1. The FSM is in Figure 13, and one frame cycle by cycle is in Figure 14.

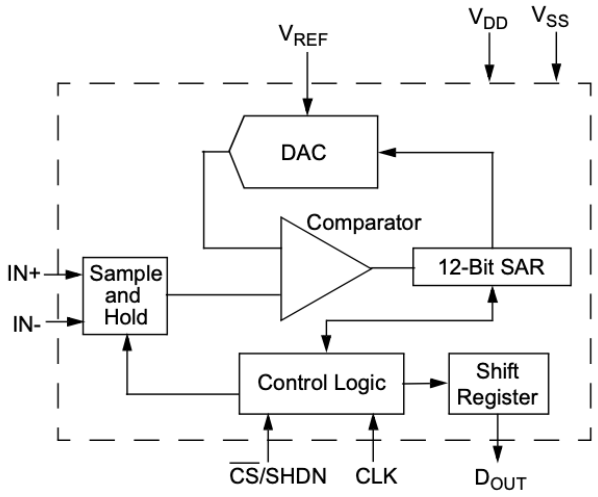

Figure 12: MCP3201 internal block diagram.

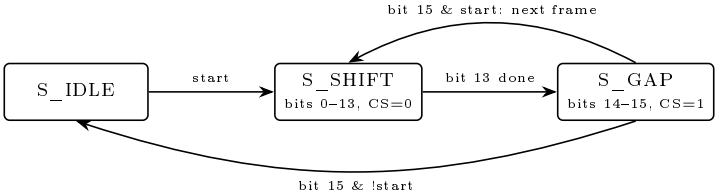

Figure 13: ADC SPI master FSM (spi_master.vhd).

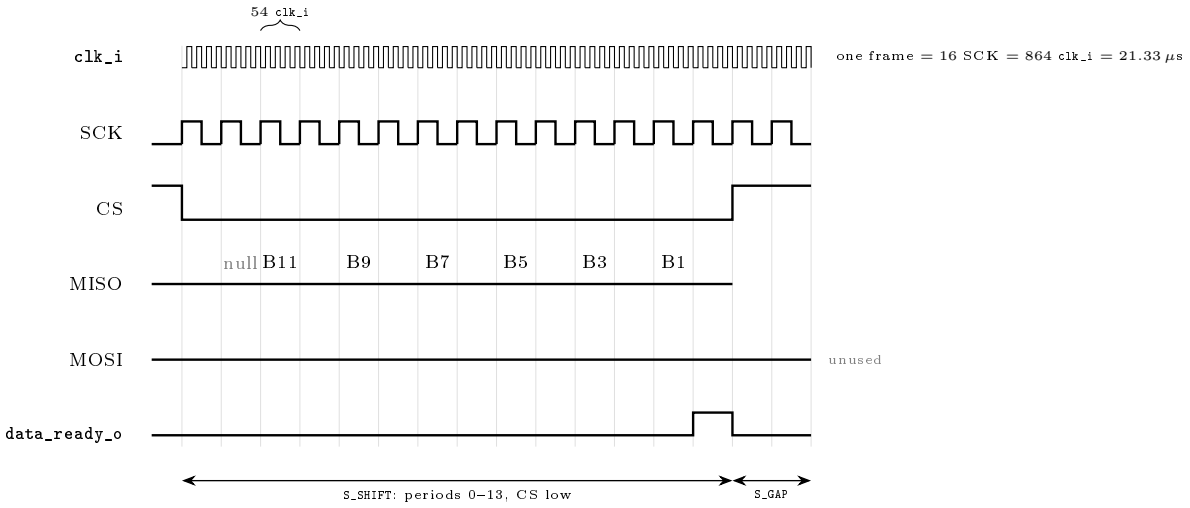

Figure 14: One MCP3201 frame, cycle by cycle.

6.3 Timing of the whole chain

The ADC sets the timing of the whole chain: one sample every 864 cycles (21.33 μs), one window of 512 samples every

$$t_{\text{window}} = 16 \times 512 \times 54 = 442\,368 \text{ cycles} = 10.92 \text{ ms} \quad \Rightarrow \quad 91.6 \text{ frames/s}, \qquad \Delta f = \frac{46\,875}{512} = 91.6 \text{ Hz per bin.}$$

Processing one window costs about 37 μs of FFT feed/collect, a 4.94 ms wait before the first SDRAM write 200 000 cycles, 25 μs of drain (21 bursts) and 202 μs of serial debug stream:

$$t_{\text{process}} = 5.2 \text{ ms} < t_{\text{window}} = 10.92 \text{ ms}.$$

The ping pong buffer is what lets capture and processing run at the same time (Figure 15). Almost all of t_{process} is that wait before the SDRAM write.

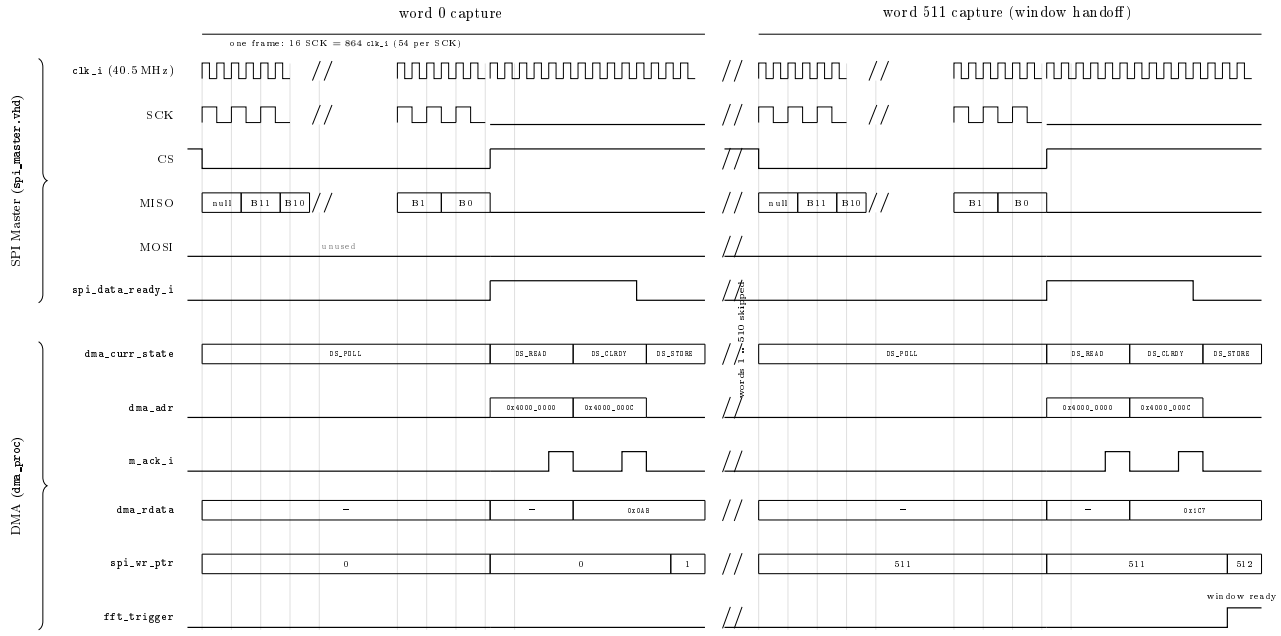

Figure 15: Capture of one 512 sample window, cycle by cycle

6.4 Fetch FSM and snapshot

The read side of the SDRAM port is the fetch FSM (Figure 16). The frame IRQ does not interrupt the CPU: in hardware it starts the fetch, which for each of the 20 bursts waits for the controller, requests a read with that burst's row address, and collects the 26 words into a 512×16 BSRAM. The write side maps each valid word to a fixed bin:

$$\text{bin} = b \times 26 + idx, \qquad b = 0 \ldots 19 \text{ (20 bursts)}, \quad idx = 0 \ldots 25 \text{ (26 words)},$$

The 20 bursts give $20 \times 26 = 520$ words, 8 more than the 512 bin BSRAM, so a bin<512 guard keeps bins 0 to 511 and drops the last 8 words of burst 19. When burst 19 finishes, fill_cnt increments. Each burst runs The whole fetch is $20 \times 200 = 4000$ cycles (about $100\,\mu$s) of the 10.92 ms frame.

The fetch fills this snapshot BSRAM, which the CPU reads. fill_cnt is how the CPU knows a new spectrum is ready: it is a counter that goes up by one each time a fetch finishes, and wraps around. The CPU keeps the last value it read and compares: when fill_cnt has changed, a fresh 512 word snapshot is ready and the CPU copies it through the Wishbone bus.

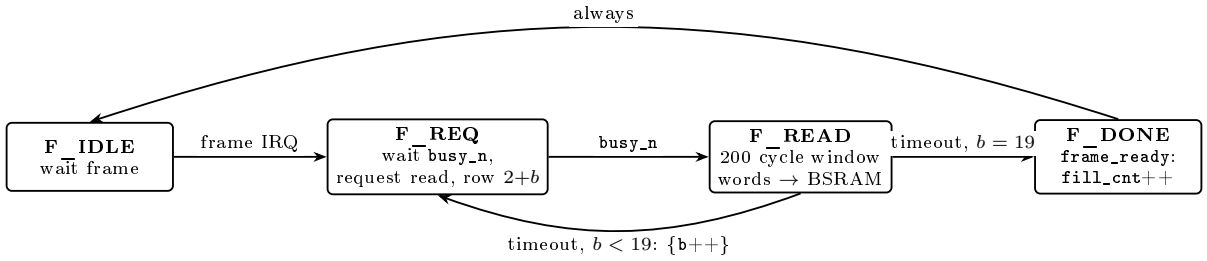

Figure 16: Fetch FSM.

6.5 The tone decoder

The decoder run in C on the soft core, it apply three tests to each of the five protocol bins: a local maximum test against the six neighbours within ± 3 bins, a curvature test on $k-1, k, k+1$ (the peak must open downward, so k is a crest and not the side of a nearby peak); and an amplitude test on the parabolically interpolated peak against a fixed threshold exactly how my thesis described.

7 Tone output

The tone output is a Sinusoidal PWM (pwm_generic_master.vhd). A free running 8 bit counter (pwm_cnt_s) counts from 0 to 255 and then wraps back to 0, so it draws a sawtooth ramp (not a triangle). A comparator turns this into the pin: PWM_o is high while pwm_cnt_s < duty_env_s, so the width of each pulse is set by the 8 bit duty (0 to 255). The counter wraps every 256 clocks, so the switching rate, the carrier, is $f_{\text{clk}}/2^8 = 40.5\,\text{MHz}/256 = 158.2\,\text{kHz}$ for the speaker, far above the audio band.

To keep the tone clean, I did not use a square wave, because a square wave also carries strong odd harmonics at $3f$, $5f$, $7f$ and so on. For a tone at $f = 2472\,\text{Hz}$ these harmonics all fall below Nyquist ($f_s/2 = 23.4\,\text{kHz}$), so on the receiver they land in other FFT bins and read as tones that were never sent.

To solve this, I implemented a PWM approach that moves the switching far away: the pin still toggles like a square, but at the fixed 158.2 kHz carrier, and the tone is carried by the duty. The duty traces a sine from the DDS and the SINE_LUT (Figure 17), and the speaker cannot follow 158.2 kHz, so it plays back a clean sine at f. The switching noise of the PWM sits around the carrier, 158.2 kHz, and its multiples, all far above the audio band, so the speaker does not reproduce it and the audio band is left with only the wanted tone.

When the module receives the address 0x04 from the bus, it loads two 16-bit frequency values. Each frequency drives its own 24-bit phase accumulator. To find how big the phase step ($\Delta\phi$) should be I would need to divide by the clock frequency, but a hardware divider is costly on an FPGA. So, using some online source (`https://electronics.stackexchange.com/questions/55493/fixed-point-division-in-verilog-for-spartan-6`), instead of dividing I multiply by a precomputed constant K and then shift the bits to the right, which gives the same result: in my code this is $(f \cdot K + 2^{23}) \gg 24$.

At every single 40.5 MHz clock tick, both 24-bit accumulators add their specific step ($\Delta\phi$) to their running total. However, the SINE_LUT only has 256 entries, meaning it only needs an 8-bit index to be read ($2^8 = 256$). So the system uses only the top 8 bits of the 24-bit accumulator to pick the sample in the table. The other 16 bits are the part after the point: they do not choose a sample, they keep track of how far the phase has moved between one sample and the next. This is why the accumulator is 24 bit and not 8: the 8 top bits pick the sample, but the full 24 bits are what let me set the frequency in fine steps. The frequency step is $f_{\text{clk}}/2^{24}$, a few Hz; with only 8 bits the smallest step would be $f_{\text{clk}}/2^8$, which is the 158 kHz carrier itself, far too coarse for an audio tone. Since FPGA hardware can run parallel tasks, the code reads two different values from the exact same table at the exact same time. To send these two different frequencies to a single output pin, the code adds the two retrieved sine values together and divides the result by 2 (a 1-bit right shift). This creates a single 8-bit mixed duty cycle without overflowing.

Finally, to make the audio sound good, the amplitude is never switched on or off abruptly. If I did that, the speaker would produce a loud clicking noise. Instead, I created a state machine that acts as a volume envelope (states E_IDLE, E_ATTACK, E_ON, E_RELEASE): it slowly increases the signal (attack) and decreases it (release) over 4 ms. During the release phase, the accumulators keep running normally so the tone fades out naturally. This final duty cycle drives a low-side IRLZ44N MOSFET, which turns the 12 V power on and off for the 8 Ω speaker. The firmware is responsible for the timing, normally sending 144 ms of tone followed by 400 ms of silence for each symbol (Section 13).

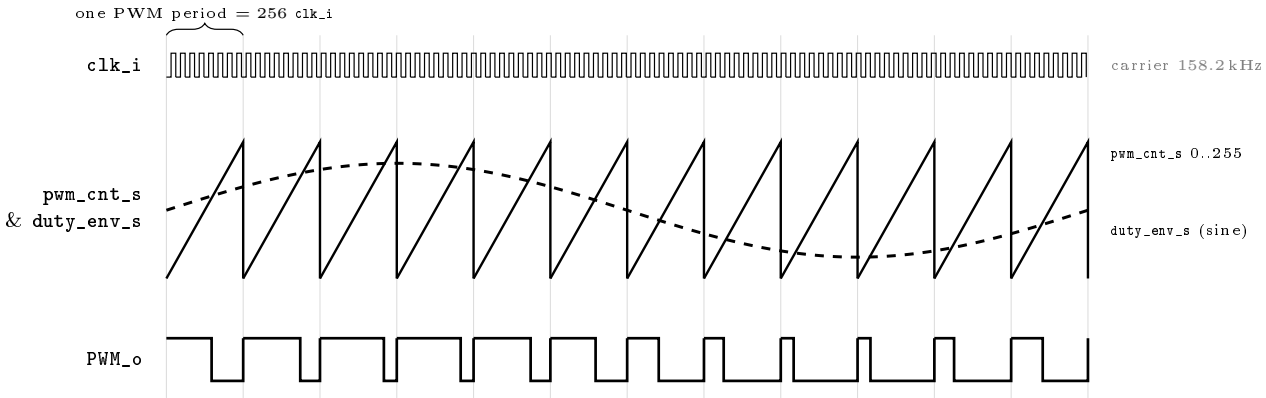

Figure 17: Sinusoidal PWM (pwm_generic_master.vhd).

8 Display

The display is an ST7789 TFT panel (240 × 320 pixels, RGB565 colour) driven over SPI. The FPGA block spi_display.vhd is a transmit only SPI in mode 3 (SCK idles high, the panel reads each bit on the rising edge). Only three wires carry data: SCK (the clock), SDA (the bits) and CS (chip select). The clock uses a ÷6

prescaler, so the SCK is 40.5 MHz/6 = 6.75 MHz. In clock cycles, one SCK is 6 clk_i and one byte is $8 \times 6 = 48$ clk_i = 1.19 μs.

The panel also has three control pins, and the firmware sets them as simple register bits: DC tells the panel whether a byte is a command or pixel data, RST resets the panel, and CS picks it. The shift FSM has three states (Figure 18): it waits for a byte, then sends the 8 bits, and after the eighth bit SCK parks high, which is already the mode 3 idle, so there is no extra state between bytes.

On top of the SPI, the firmware draws a dashboard on the panel. At boot it first sends the ST7789 init sequence (SWRESET, SLPOUT, COLMOD RGB565, MADCTL landscape, INVON, NORON, DISPON), each command with its datasheet delay. After that the drawing works step by step. When the firmware wants to draw something it does not send it right away: it puts the drawing as a job in a queue that holds up to 48 jobs. Then, once per main loop, the function display_tick() takes one job from the queue and sends only a chunk of SPI bytes (128 to 384 bytes, about 150 to 450 μs). This chunk is short next to the 10.9 ms frame, so it does not slow down the tone player. If instead the firmware drew a full screen in one go, the SPI would take about 180 ms and block the main loop, which would stop the tone and fill up the command FIFO. Sending it in chunks avoids this.

Four fixed zones show slave health, the live spectrum (bins 16 to 55 straight from the snapshot), RX/TX symbols and a log ring; which content each zone shows and the three theme colours come from the NOR settings, so the web dashboard restyles the display remotely.

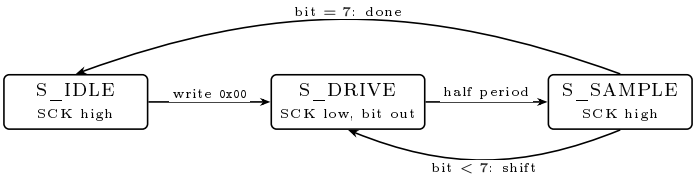

Figure 18: Display SPI FSM (spi_display.vhd).

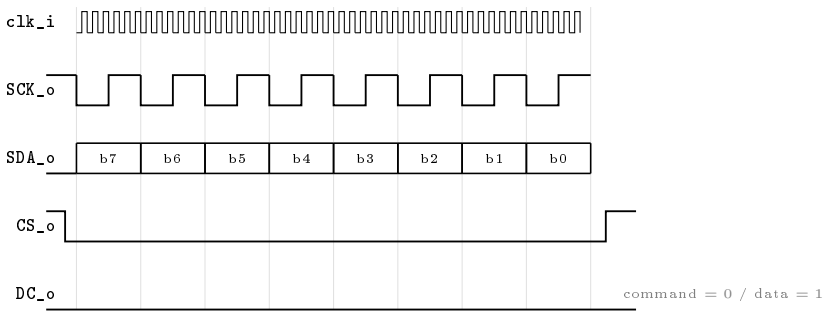

Figure 19: One display byte (spi_display.vhd).

9 NOR flash storage

The CPU manages NOR storage in firmware (norflash.c). The ESP32 sends one character opcodes on the S7 UART (pins 72/20), and nor_poll(), a non blocking step in the main loop, reads each opcode and runs the matching W25Q64 SPI sequence; the SPI frames go out through the generic S8 master at 633 kHz. At power up nor_init() runs the boot chain, then the loop sits in the IDLE hub and dispatches each opcode with a case (Figure 24).

The chip uses the standard W25Q64 opcodes (Table 7). Every access is one SPI transaction: CS goes low, the master (spi_master_generic.vhd) clocks the command byte MSB first, then a 24 bit address if the command needs one, then data, and CS goes high. A read (0x03) sends the address and the chip drives the bytes back on MISO (Figure 20). A write is longer: a Write Enable (0x06) frame first, then a Page Program (0x02) with the address and the data, then the master polls the status register (0x05) until the WIP bit is 0 (Figure 21). An erase works like the program, with 0x20 and no data.

In clock cycles: SCK is clk_i/64 = 633 kHz, so one SCK is 64 clocks and one byte is $8 \times 64 = 512$ clocks (12.6 μs). A read is the command byte plus the 3 address bytes ($4 \times 512 = 2048$ clocks = 50.6 μs) and then one more byte per data byte read. A page program is the WREN byte, then the command, 3 address bytes and up to 256 data bytes, so 260 SPI bytes; after that the status poll runs until the WIP bit clears, and that wait is set by the chip's internal program time, not by SPI clocks.

Table 7: W25Q64 SPI commands used.

Code	Name	Use
0x06	Write Enable	set before every program or erase
0x01	Write Status Register	set QE, clear the protection bits
0x98	Global Block Unlock	clear all block protection
0x05	Read Status Register	polled for the WIP (busy) bit
0x9F	JEDEC ID	read manufacturer and device id
0x03	Read Data	read bytes from a 24 bit address
0x02	Page Program	write up to 256 bytes at a 24 bit address
0x20	Sector Erase	erase one 4 KB sector

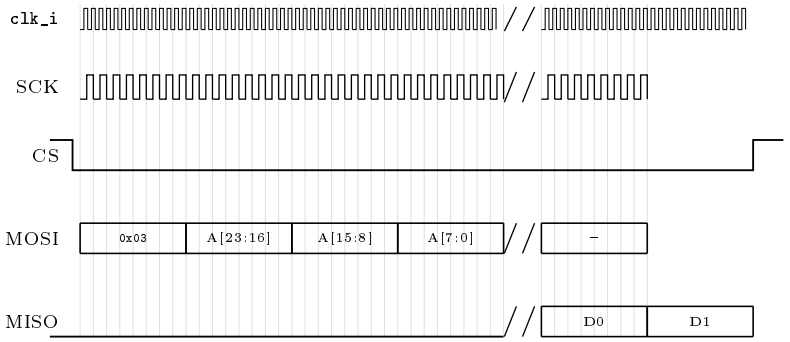

Figure 20: NOR flash read 0x03 (spi_master_generic.vhd).

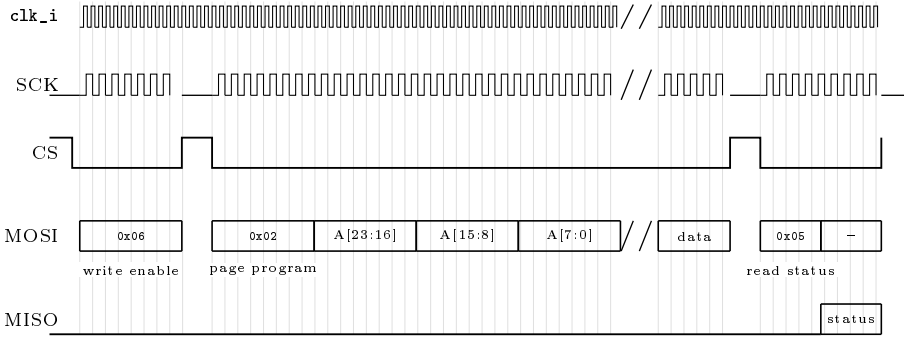

Figure 21: NOR flash page program 0x02 (spi_master_generic.vhd).

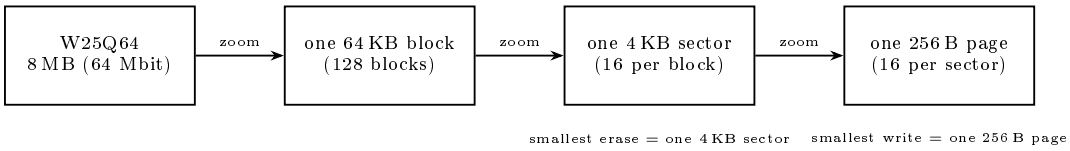

Figure 22: W25Q64 layout: blocks, sectors, pages.

Memory layout. The W25Q64 is built from 64 KB blocks, each split into 4 KB sectors, each split into 256 B pages (Figure 22); the smallest erase is one sector and the smallest write is one page. The firmware uses only the first 12 KB (Figure 23): sector 0 (4 KB) holds a 32 byte settings blob that starts with a fixed byte 0xA5 (a marker the firmware checks to know the settings are valid; if it is missing it uses defaults), then the device name, auto presence interval, retry count, debug flag, display theme colours, view flags, v2 marker 0x5A); sectors 1 to 2 hold the log, 512 slots of 16 bytes (sequence, type character, timestamp, value, 8 ASCII bytes) managed as a ring buffer. At power up a SCAN reads the type byte of all 512 slots and takes head = (highest valid +1) mod 512; treating anything outside the valid type set as empty makes the scan ignore the gaps an aborted write leaves.

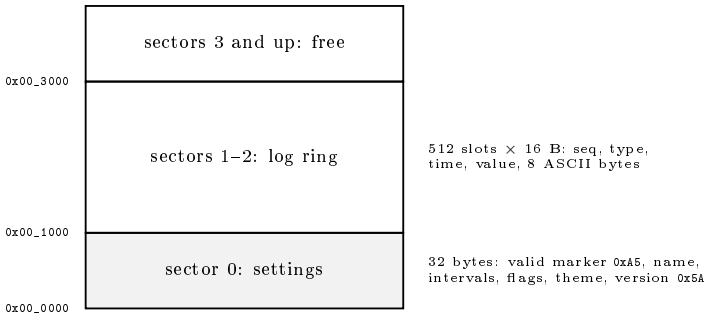

Figure 23: How the firmware organises the flash.

Unlock before reading the id. On this board the /HOLD and /WP pins are left unconnected. A floating /HOLD pin picks up noise from the SCK line, which makes the chip stop in the middle of a transfer, and then every read gives 0xFF, even the chip id. So I cannot read the id first to check that the chip is there. Instead, at boot the firmware first sends a WRSR command that sets the QE bit (this frees the /HOLD and /WP pins) and clears the write protection, then a Global Block Unlock, and only after that it reads the id. If the id is still wrong, the firmware marks the chip as missing, so the later commands fail quickly instead of waiting a long time. The timeouts are measured in real time, not in loop counts: 2 ms per SPI byte, 700 ms for the busy poll (an erase can take 400 ms), and 200 ms per byte coming from the ESP32.

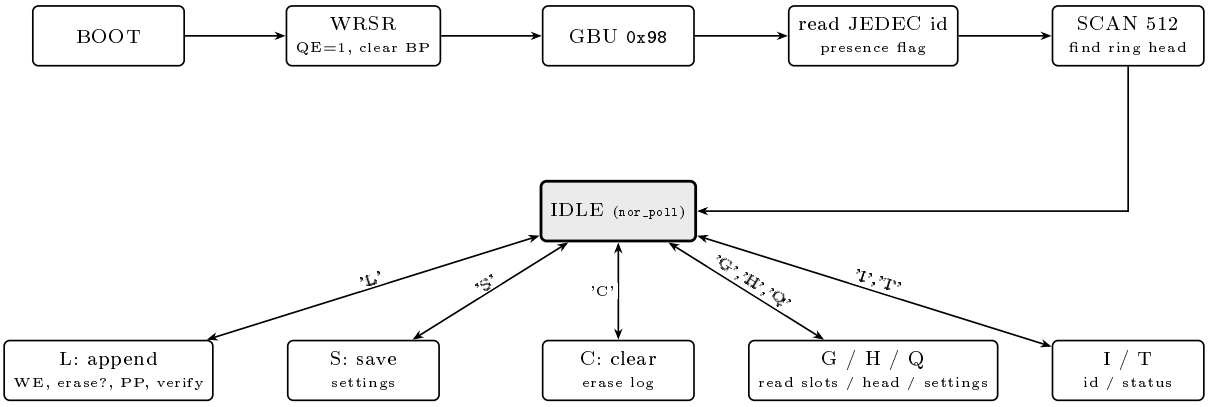

Figure 24: Storage FSM (norflash.c).

Board notes. During a sector erase the chip draws a short current burst, and on the bare wiring this made the 3.3 V rail drop enough to reset the chip in the middle of the erase. To fix it I added two capacitors in parallel at the chip, one 1 μF and one 10 μF.

10 The UART links

The FPGA has two identical UARTs from uart_generic.vhd: the character UART (S6, pins 17/18) that carries diagnostics and banner text, and the NOR command UART (S7, pins 72/20) that carries the one character flash opcodes (the full set is in Figure 24). They are two instances of the same block with the same register map (Table 2): 0x00 writes a byte to send and reads the received byte (bit 8 is the valid flag), 0x04 and 0x08 enable and disable, 0x0C sets the baud divider and 0x10 the format, and 0x14 is the status (rx_valid, tx_busy). At 115 200 baud the divider is 352, so one bit lasts 352 clocks.

Each instance has a transmit and a receive state machine (Figure 26). On transmit, a write to 0x00 sets tx_busy and the machine sends a start bit, 8 data bits LSB first, an optional parity bit and a stop bit (Figure 25). On receive, two flip flops (rx_q0, rx_q1) synchronise the line; a falling edge starts the machine, it samples the first bit at half a bit time and the rest one bit time apart, and each finished byte goes into a 16 deep FIFO (rxf) that the master reads from 0x00.

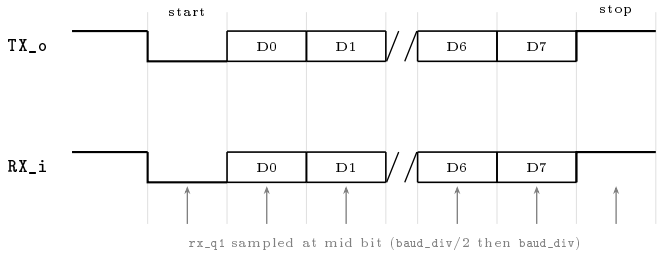

Figure 25: UART frame, 8N1 (uart_generic.vhd). One bit is baud_div clocks; data goes LSB first.

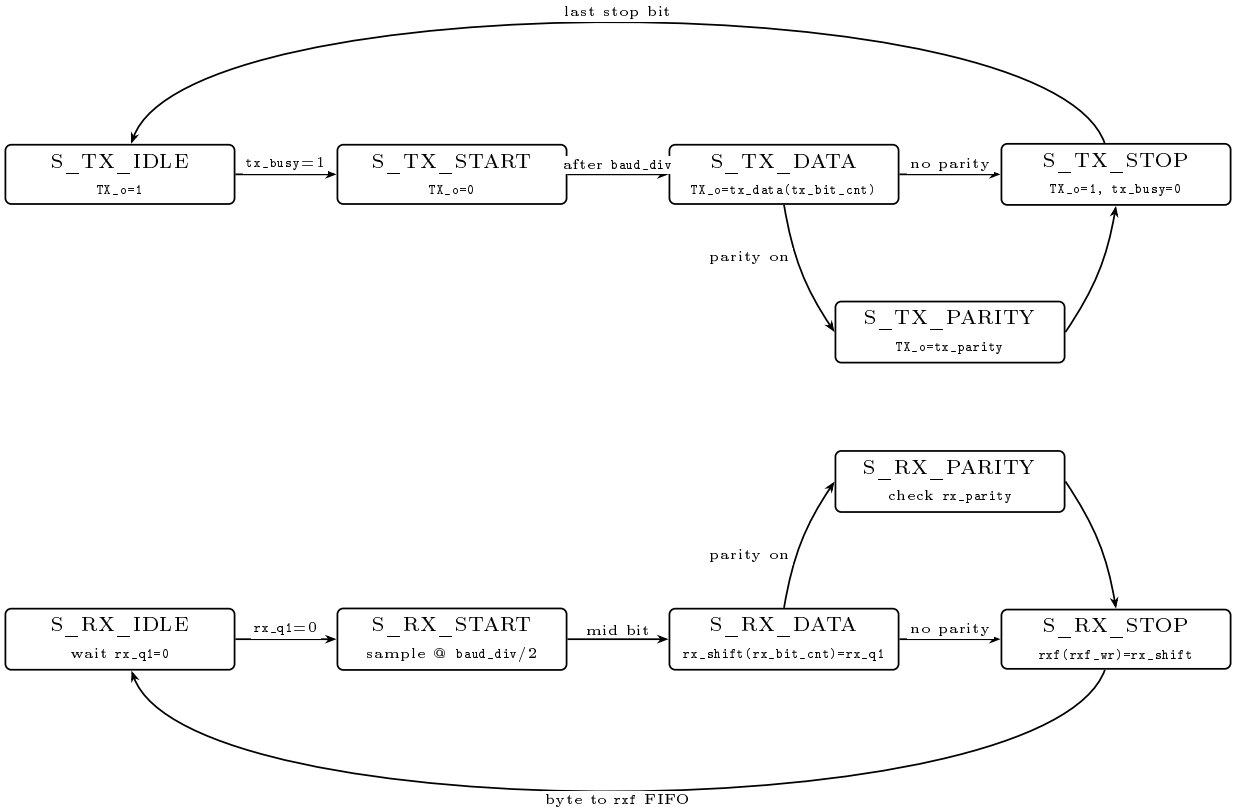

Figure 26: UART transmit and receive state machines (uart_generic.vhd).

11 Firmware

The soft core runs a firmware written in C (seven files plus a startup file), built for rv32imc. The code sits in the 32 KB ITCM and the data in the 16 KB DTCM, both local to the CPU and off the bus. The build turns the binary into a hex file that the CPU loads into the ITCM at start up, so the firmware travels inside the FPGA bitstream and needs no external memory.

The firmware is one main loop. Each pass of the loop calls a few short functions, one after the other, always in this order (Figure 27). First emit_capture takes the characters that arrived from the ESP32 and puts them in a queue. Then emit_player_tick takes one character from the queue and plays its tone pair on the speaker PWM (144 ms of tone, then 400 ms of silence); it also listens, so if it has heard the other device in the last 3 s it holds the queue, and if the other device starts talking in the middle of a tone it stops the PWM at once. Next decode_step checks fill_cnt, and when a new spectrum is ready it copies the snapshot, decodes it, and sends the decoded characters to the ESP32. After that nor_poll runs the flash storage protocol (Section 9), and finally display_tick sends one small chunk to the display (Section 8).

A health_tick runs every 15 s to check that the slaves are alive. It reads one safe register from each slave and looks at the value it gets back; if a slave gives a wrong or stuck value it is marked as not healthy. The result is packed into a short status word ($HLT) that both the display and the web dashboard show, so I can see at a glance which blocks are working. The same idea also helped at boot: just before the firmware talks to a slave for the first time, it prints a marker on the diagnostics UART. If that slave locks up the CPU, the last

marker printed is the name of the slave that caused it, so I know straight away where the problem is. This is how I found the GPIO ack bug in a few minutes.

Diagnostics use the character UART. They are simple lines that start with $ and end with a newline; the ESP32 sends these to the USB console instead of to the protocol parser. A debug flag in the NOR settings, which can be switched from the dashboard, turns them on and off.

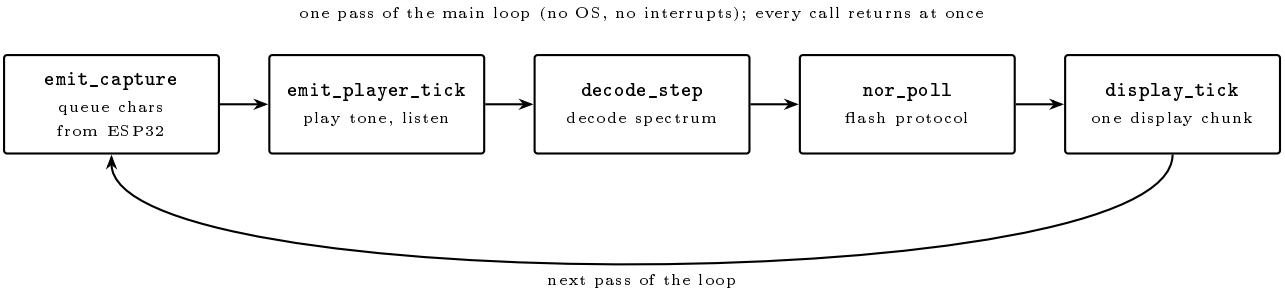

Figure 27: The firmware main loop.

12 The ESP32

The ESP32 handles the protocol logic, the queue of incoming and outgoing messages, and the link with the web interface. It does not touch the FPGA logic directly: the two chips talk only over two UART links (Figure 28), and they share a common ground.

The first link is the character UART (FPGA pins 17 and 18): the FPGA sends the decoded protocol characters and the diagnostics, and the ESP32 sends back the characters to transmit. The second link is the NOR command UART (FPGA pins 72 and 20): the ESP32 sends the one character flash opcodes and the FPGA firmware runs them. Both links are 8N1 at 115 200 baud.

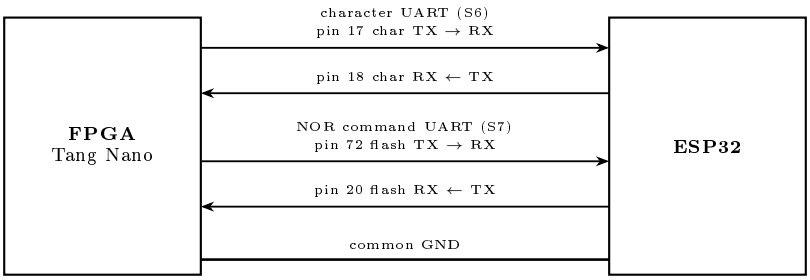

Figure 28: ESP32 to FPGA: two UART links and a shared ground.

The character UART carries the protocol symbols (both ways) and the diagnostics, while the NOR command UART carries the one character flash opcodes. Table 8 lists the bytes that go over each link.

Table 8: Bytes exchanged on the two links.

Link	Byte	Meaning
character UART	protocol symbols	the tones, decoded or to send (Section 13)
character UART	$... line	diagnostics (FPGA to ESP32)
NOR command UART	L	append a log record
NOR command UART	S	save the settings
NOR command UART	C	clear the log
NOR command UART	G	read log slots
NOR command UART	H	read the ring head
NOR command UART	Q	read the settings
NOR command UART	I	read the chip id
NOR command UART	T	read the status

On the ESP32 side the software is a few modules: a UART driver that writes the chip registers directly (115 200 baud from the 80 MHz clock), a flash link with retry and read back, the codebook, the protocol state machine, and the WebSocket client for the dashboard. The Wi-Fi transmit power is kept at 11 dBm, because at full power the current bursts once tripped the brownout detector on a weak supply.

13 The acoustic protocol

The protocol uses an alternating bit codebook. A pattern is two or three symbols in which two consecutive symbols must differ, so the receiver triggers on a change of tone, easy to see, rather than on silence, which is hard to measure on a noisy channel. Six symbols exist: lowercase a,b,c (bit 0, bit 1, EOF) ride the 2014 Hz master carrier (FFT bin 22), uppercase A,B,C ride the 2472 Hz slave carrier (bin 27); EOF, bit 0 and bit 1 use 2930, 3571 and 4577 Hz (bins 32, 39, 50), all exact bin centres of the shared 91.6 Hz grid, the closest pair 7 bins apart. A message is identified by its first bit and length, the carrier gives the direction (Table 9); every pattern is framed by EOF at both ends.

Table 9: The codebook.

Pattern	(first, len)	Master carrier (to slave)	Slave carrier (to master)
01	(0, 2)	presence request	presence: yes
10	(1, 2)	assign id (= 1)	presence: no
010	(0, 3)	abort	registration request
101	(1, 3)	,	acknowledge

On the air, each transmission adds three things (Figure 29). First, a warm up symbol: the EOF tone is held for 300 ms instead of 144 ms, which gives the receiving microphone time to settle; it carries no data. Second, the EOF that frames the message is sent four times in a row, because losing an EOF breaks the whole message, while losing a data bit only loses one bit. Third, each symbol is 144 ms of tone followed by 400 ms of silence; a shorter gap would let the echo of one tone run into the next and merge two equal bits into one. The receiver does the opposite: it only accepts a decoded symbol if it is different from the last one it accepted, which turns the repeated tones and the four EOFs back into single symbols. If a message is left half received with no new symbol for 2 s, it is dropped.

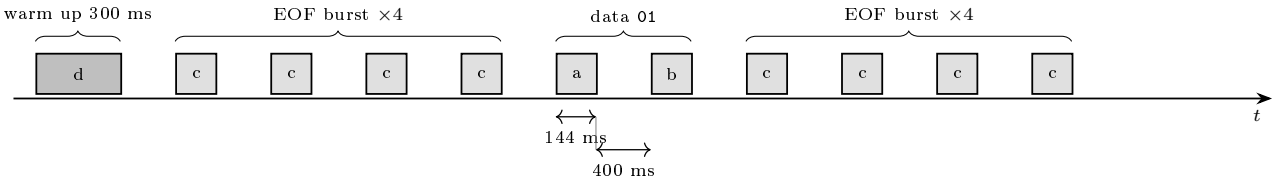

Figure 29: One transmission on the air.

Only one device talks at a time. The master checks this in two ways. First, the soft core listens: if it hears the other device, it holds the tone player and stops a tone that is playing. Second, before sending each byte the ESP32 waits for 1 s of silence, and it gives up after 15 s so a noisy line cannot block sending forever. The channel loses tones often, so both sides retry: a presence request with no answer for 8 s is sent again up to a set number of times (default 6, saved in flash), and the registration message is sent again on a random 30 to 40 s timer until it is confirmed. Receiving the same message twice does no harm, so repeats are safe.